🏗️ ARCHITECTURE

HelixCode Architecture Documentation

System Overview

HelixCode is a distributed AI development platform designed for enterprise-grade scalability and reliability. The architecture follows microservices principles with clear separation of concerns and robust distributed computing capabilities.

Core Architecture Components

1. Distributed Worker Network

SSH-Based Worker Pool

- **SSHWorkerPool**: Manages SSH-accessible worker nodes
- **Worker Registration**: Automatic discovery and registration
- **Health Monitoring**: Real-time worker health checks
- **Capability Detection**: Automatic hardware and software capability detection
- **Auto-Install**: Automatic Helix CLI installation on worker nodes

Worker Management

- **WorkerManager**: Central worker lifecycle management
- **Resource Allocation**: Dynamic resource allocation based on capabilities
- **Load Balancing**: Intelligent task distribution across workers
- **Failure Recovery**: Automatic worker recovery and task reassignment

2. Advanced LLM Integration

Provider Architecture

```
type LLMProvider interface {
    Generate(ctx context.Context, req *LLMRequest)
        (*LLMResponse, error)
    GenerateStream(ctx context.Context, req *LLMRequest, ch
        chan<- LLMResponse) error
```

```
    GetModels() []Model
    GetCapabilities() []ModelCapability
    GetHealth(ctx context.Context) (*ProviderHealth, error)
    IsAvailable(ctx context.Context) bool
    Close() error
}
```

Supported Providers

- **Local Models**:
  - Llama.cpp: Direct local inference
  - Ollama: Streamlined local model management
- **Cloud Providers**:
  - **Anthropic Claude**: Extended thinking, prompt caching, tool caching, 200K context
  - **Google Gemini**: 2M token context, function calling, safety settings
  - **OpenAI**: GPT-4, GPT-3.5-turbo with function calling
  - **xAI**: Grok models with reasoning capabilities
  - **Qwen**: Chinese language models with OAuth2
- **Aggregators**:
  - **OpenRouter**: Multi-provider access with unified API
  - **GitHub Copilot**: GitHub integration for multiple models

Advanced Features

- **Extended Thinking**: Automatic reasoning mode for complex tasks (Anthropic)
  - Keyword-based detection
  - 80% token budget allocation
  - Transparent thinking process
- **Prompt Caching**: Multi-layer caching for cost optimization (Anthropic)
  - System message caching (5-minute TTL)
  - Conversation history caching
  - Tool definition caching
  - Up to 90% cost reduction
- **Massive Context**: 2M token context windows (Gemini)
  - Full codebase analysis
  - Long-form documentation processing
  - Complex multi-file reasoning
- **Function Calling**: Structured tool integration
  - AUTO mode: Automatic tool selection
  - ANY mode: Force tool usage
  - NONE mode: Disable tools

- **Vision Capabilities**: Image understanding (Anthropic, Gemini)
    - Code screenshot analysis
    - Diagram interpretation
    - UI/UX review
- **Streaming**: Real-time response generation
    - Server-Sent Events (SSE)
    - Chunk-based updates
    - Progress indicators

Reasoning Engine

- **Chain-of-Thought**: Step-by-step reasoning with intermediate results
- **Tree-of-Thoughts**: Multiple reasoning paths with selection
- **Self-Reflection**: Error correction and improvement cycles
- **Progressive Reasoning**: Incremental reasoning with tool integration
- **Extended Thinking**: Deep reasoning with transparent thought process

3. MCP (Model Context Protocol) Integration

Protocol Support

- **Stdio Transport**: Process-based communication
- **SSE Transport**: Server-Sent Events for real-time updates
- **HTTP Transport**: RESTful API communication
- **WebSocket Transport**: Bidirectional real-time communication

Tool Discovery & Management

- **Dynamic Tool Registration**: Runtime tool discovery
- **Multi-Server Support**: Concurrent MCP server management
- **Authentication**: OAuth2 and API key support
- **Resource Management**: Efficient resource allocation and sampling

4. Multi-Client Architecture

Client Types

- **REST API**: Comprehensive HTTP API with OpenAPI specification
- **Terminal UI**: Rich interactive terminal interface
- **CLI**: Command-line interface for scripting and automation
- **Mobile Apps**: Native iOS and Android applications

Communication Protocols

- **HTTP/REST**: Standard RESTful API
- **WebSocket**: Real-time bidirectional communication
- **SSH**: Secure shell for worker communication
- **MCP**: Model Context Protocol for tool integration

5. Notification System

Multi-Channel Support

- **Slack**: Webhook and bot integration
- **Discord**: Bot API with rich embeds
- **Telegram**: Bot API with media support
- **Email**: SMTP with HTML templates
- **Yandex Messenger**: Russian platform integration
- **Max**: Enterprise communication platform

Notification Engine

- **Rule-Based Routing**: Configurable notification rules
- **Template System**: Customizable message templates
- **Priority System**: Priority-based delivery
- **Fallback Strategies**: Multi-channel fallback

Database Architecture

PostgreSQL Schema

Core Tables

- **users**: User authentication and profiles
- **user_sessions**: Active user sessions
- **workers**: Distributed worker nodes
- **worker_metrics**: Performance metrics collection
- **distributed_tasks**: Task management with work preservation
- **task_checkpoints**: Automatic checkpointing system
- **projects**: Project management
- **sessions**: Development sessions
- **notifications**: Notification system
- **mcp_servers**: MCP server configurations
- **llm_models**: LLM model management

Work Preservation Features

- **Automatic Checkpointing**: Periodic task state saving
- **Dependency Management**: Task dependency tracking
- **Criticality Levels**: Task importance classification
- **Rollback System**: Automatic rollback on failures
- **Graceful Degradation**: System stability during failures

Security Architecture

Authentication & Authorization

- **JWT-Based Authentication**: Secure token-based authentication
- **Role-Based Access Control**: Fine-grained permission system
- **Multi-Factor Authentication**: Enhanced security options
- **Session Management**: Secure session handling

Encryption & Security

- **End-to-End Encryption**: All communications encrypted
- **Secure Key Management**: Proper key rotation and storage
- **Input Validation**: Comprehensive input sanitization
- **Security Headers**: HTTP security headers

Performance & Scalability

Performance Targets

- **Response Time**: <500ms for all operations
- **Resource Efficiency**: >85% hardware utilization
- **Scalability**: Support for 100+ concurrent workers
- **Availability**: 99.9% uptime for core features

Scalability Features

- **Horizontal Scaling**: Worker pool expansion
- **Load Balancing**: Intelligent task distribution
- **Distributed Caching**: Efficient state management
- **Resource Optimization**: Dynamic resource allocation

Deployment Architecture

Container-Based Deployment

```yaml
services:
  helixcode-server:
    image: helixcode/server:latest
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/helixcode
      - REDIS_URL=redis://redis:6379
    ports:
      - "8080:8080"

  worker-node-1:
    image: helixcode/worker:latest
    environment:
      - HELIX_SERVER_URL=http://helixcode-server:8080
      - WORKER_CAPABILITIES=llm-inference,code-generation
```

High Availability Setup

- **Database Replication**: PostgreSQL streaming replication
- **Load Balancer**: Round-robin worker distribution
- **Health Checks**: Comprehensive system monitoring
- **Backup Strategy**: Automated backup and recovery

Monitoring & Observability

Metrics Collection

- **System Metrics**: CPU, memory, disk usage
- **Application Metrics**: Request rates, error rates, response times
- **Business Metrics**: User activity, task completion rates
- **Worker Metrics**: Health status, performance metrics

Logging Strategy

- **Structured Logging**: JSON-formatted logs
- **Log Levels**: Debug, Info, Warn, Error
- **Log Aggregation**: Centralized log collection
- **Audit Logging**: Security and compliance logging

Development Workflows

Distributed Development Modes

Planning Mode

- Distributed project analysis
- Multi-source technology research
- Architecture design with collaborative input
- Resource requirement calculation

Building Mode

- Distributed compilation and building
- Parallel code generation
- Build artifact caching
- Cross-platform build support

Testing Mode

- Distributed test execution
- Parallel test suites
- Comprehensive quality scanning
- Performance testing across workers

Refactoring Mode

- Distributed refactoring operations
- Cross-file refactoring coordination
- Safety validation and rollback
- Collaborative refactoring sessions

Integration Patterns

External Service Integration

- **LLM Providers**:
  - Local: Llama.cpp, Ollama
  - Cloud: Anthropic Claude, Google Gemini, OpenAI, xAI, Qwen
  - Aggregators: OpenRouter, GitHub Copilot
- **Version Control**: Git integration
- **CI/CD Systems**: Jenkins, GitHub Actions
- **Monitoring Tools**: Prometheus, Grafana

Plugin System

- **Extension Points**: Well-defined extension interfaces
- **Hot Reloading**: Runtime plugin loading
- **Dependency Management**: Plugin dependency resolution
- **Security Sandboxing**: Secure plugin execution

Future Architecture Evolution

Planned Enhancements

- **Edge Computing**: Edge device integration
- **Federated Learning**: Distributed model training
- **Blockchain Integration**: Immutable task tracking
- **Quantum Computing**: Quantum algorithm support

Scalability Roadmap

- **Microservices**: Further service decomposition
- **Event-Driven Architecture**: Event sourcing implementation
- **Service Mesh**: Advanced service communication
- **Multi-Region Deployment**: Global distribution

Architecture Version: 1.1.0 **Last Updated**: 2025-11-05 **Compatibility**: Go 1.26+, PostgreSQL 15+, Redis 7+

Recent Architecture Changes (v1.1.0)

New LLM Provider Integrations

- **Anthropic Claude Provider**: Full API implementation with advanced features
  - Extended thinking with automatic detection
  - Multi-layer prompt caching (system/messages/tools)
  - Tool caching for repeated operations
  - Vision support for image analysis
  - Streaming with Server-Sent Events
- **Google Gemini Provider**: Complete API integration with massive context support
  - 2M token context windows (Gemini 2.5 Pro, 1.5 Pro)
  - Function calling with AUTO/ANY/NONE modes
  - Configurable safety settings
  - System instruction separation

    - Vision and multimodal capabilities

Enhanced Provider Architecture

- Unified provider interface for all LLM backends
- Health monitoring and availability checks
- Model capability introspection
- Streaming support across all providers
- Error handling with context-aware retries